

厦门大学林子雨编著

《Python 程序设计基础教程（微课版）》

教材习题 答案

第 7 章 面向对象程序设计

（版本号：2022 年 1 月版本）



主讲教师：林子雨
厦门大学数据库实验室
二零二二年一月

一、简答题

1. 简单描述保护属性安全性的一般处理方式。

【参考答案】

把这个属性设置为私有的，并设置相应的读写方法。

2. 简单描述什么是多重继承。

【参考答案】

一个子类有多个直接父类。

3. 简单描述什么是方法的重写。

【参考答案】

子类采用同样的方法签名修改继承自父类的方法。

二、编程题

1. 定义一个类判断给定字符串中的括号（包括“<>”、“（）”、“[]”、“{}”四种类型的括号）是否正确配对。例如：“()adafd{}”是错误配对，“()adafd{(<>)}"是正确配对，“{[{()]}"是正确配对。

【参考答案】

```
class Solution:

    def is_valid(self, str1):

        stack= []

        parenthese_dict = {"<": ">", "(": ") ", "{": " }", "[": " ]"}

        for c in str1:

            if c in parenthese_dict.keys():

                stack.append(c)

            elif c in parenthese_dict.values():

                if len(stack)==0 or parenthese_dict[stack.pop()] !=

c:

                    return False

            return len(stack) == 0

if __name__ == '__main__':

    print(Solution().is_valid("()adafd{}")) # 输出False

    print(Solution().is_valid("()adafd{(<>)}")) # 输出True

    print(Solution().is_valid("{[{()]})")) # 输出True
```

2. 实现一个名为“Rectangle”的表示矩形的类，该类包含两个公有的实例属性 width 和 height，分别表示矩形的宽和高，同时还有一个名为 aera 的公有方法，该方法返回矩形的面积。

【参考答案】

```
class Rectangle:

    def __init__(self,width,height):

        self.width = width

        self.height = height

    def area(self):

        return self.width * self.height
```

3.修改习题 2 中的 Rectangle 类，将实例属性 width 和 height 改为可读写的 property，并且在写操作时检查是否为正值，将 area 方法改为一个只读的 property。下面是测试代码：

```
>>> a = Rectangle(3,4)

>>> a.width = -5 # 输出错误提示

>>> a.width = 5

>>> a.height = 0 # 输出错误提示

>>> a.height = 2

>>> a.area    # 返回10

>>> a.area = 12 # 输出“AttributeError: can't set attribute”
```

【参考答案】

```
class Rectangle:

    def __init__(self,width,height):

        self._width = width # 为了实现后文的继承，将属性设为保护型的

        self._height = height

    @property

    def width(self):

        return self._width

    @width.setter

    def width(self,width):

        if (width<=0):

            print('width必须为正值,设置失败')

        else:

            print('width设置成功')
```

```
        self._width = width

@property
def height(self):
    return self._height

@height.setter
def height(self,height):
    if(height<=0):
        print('height必须为正值,设置失败')
    else:
        print('height设置成功')
        self._height = height

@property
def area(self):
    return self.width * self.height
```

4.创建一个 User 类，其包含一个 name 的实例属性，由构造函数初始化，还包含一个名为 count 的类属性用于统计用户数目（创建的实例数）。下面是测试代码：

```
>>> a = User('Mike')

>>> User.count # 输出1

>>> b = User('Tom')

>>> User.count # 输出2

>>> c = User('Jack')

>>> User.count # 输出3
```

【参考答案】

```
class User(object):
    count=0
    def __init__(self,name):
        self.name = name
        User.count += 1
```

5. 定义一个名为 `Calculator` 的类，使用静态方法定义四个表示加减乘除的运算。测试代码如下：

```
>>> Calculator.add(10, 5) # 返回 15

>>> Calculator.subtract(10, 5) # 返回 5

>>> Calculator.multiply(10, 5) # 返回 50

>>> Calculator.divide(10, 5) # 返回 2
```

【参考答案】

```
class Calculator:

    @staticmethod
    def add(x, y):
        return x+y

    @staticmethod
    def subtract(x, y):
        return x-y

    @staticmethod
    def multiply(x, y):
        return x*y

    @staticmethod
    def divide(x, y):
        return x/y
```

6. 请修改下面的 `Point` 类，通过重写相应魔法方法使其支持如下运算符操作：

`p1+p2`, 用两个点的 `x`、`y` 坐标分别相加，返回一个新的 `Point` 对象；

`p1-p2`, 用两个点的 `x`、`y` 坐标分别相减，返回一个新的 `Point` 对象；

`p*n`, 用点的 `x`、`y` 坐标分别乘以数值 `n`，返回一个新的 `Point` 对象；

`p/n`, 用点的 `x`、`y` 坐标分别除以数值 `n`，返回一个新的 `Point` 对象。

`class Point:` # 请直接修改该类的定义

```
def __init__(self, x, y):

    self.x = x

    self.y = y
```

```
def __str__(self):  
    return 'Point('+str(self.x)+','+str(self.y)+')'  
  
if __name__ == '__main__': # 这是测试代码  
    a,b=Point(4,5),Point(2,3)  
    print(a+b) # 输出结果为'Point(6,8)'  
    print(a-b) # 输出结果为'Point(2,2)'  
    print(a*2) # 输出结果为'Point(8,10)'  
    print(a/2) # 输出结果为'Point(2.0,2.5)'
```

【参考答案】

```
class Point:  
    def __init__(self,x,y):  
        self.x = x  
        self.y = y  
  
    def __str__(self):  
        return 'Point('+str(self.x)+','+str(self.y)+')'  
  
    def __add__(self,other):  
        return Point(self.x+other.x,self.y+other.y)  
  
    def __sub__(self,other):  
        return Point(self.x-other.x,self.y-other.y)  
  
    def __mul__(self,n):  
        return Point(self.x*n,self.y*n)  
  
    def __truediv__(self,n):  
        return Point(self.x/n,self.y/n)
```

```
if __name__ == '__main__': # 这是测试代码
    a,b=Point(4,5),Point(2,3)

    print(a+b) # 输出结果为'Point(6,8)'
    print(a-b) # 输出结果为'Point(2,2)'
    print(a*2) # 输出结果为'Point(8,10)'
    print(a/2) # 输出结果为'Point(2.0,2.5)'
```

7.请继承习题 3 中的 `Rectangle` 类来实现一个名为 `Square` 的正方形类,使得可以用 `Square`(边长)的方式实例化该类,并重写父类中 `width` 和 `height` 的写方法,实现长和高的同时修改。测试代码如下:

```
>>> a = Square(5)

>>> a.width = 10 # 同时修改height

>>> a.height # 输出 10

>>> a.height = 20 # 要求同时修改width

>>> a.width # 输出 20

>>> a.area # 输出400
```

【参考答案】

```
class Rectangle:
    def __init__(self,width,height):
        self._width = width
        self._height = height

    @property
    def width(self):
        return self._width

    @width.setter
    def width(self,width):
        if(width<=0):
            print('width 必须为正值,设置失败')
        else:
            print('width 设置成功')
            self._width = width

    @property
```

```
def height(self):
    return self._height

@height.setter
def height(self,height):
    if(height<=0):
        print('height 必须为正值,设置失败')
    else:
        print('height 设置成功')
        self._height = height

@property
def area(self):
    return self.width * self.height

class Square(Rectangle):
    def __init__(self,width):
        super().__init__(width,width)

    @Rectangle.height.setter
    def height(self,height):
        if(height<=0):
            print('height 必须为正值,设置失败')
        else:
            print('height 设置成功')
            self._height = height
            self._width = height

    @Rectangle.width.setter
    def width(self,width):
        if(width<=0):
            print('width 必须为正值,设置失败')
        else:
            print('width 设置成功')
            self._width = width
            self._height = width

b = Square(4)
print(b.width)
print(b.height)
b.width = -5 # 输出错误提示
b.width = 5
b.height = 0 # 输出错误提示
```



```
b.height = 2
print(b.area)  # 返回 4

b.area = 12 # 输出“AttributeError: can't set attribute”
```

8. 继承内建的字符串类，实现对字符串的左右循环移动给定整数位（正数代表左移，负数代表右移）。

【参考答案】

```
class MoveableStr(str):

    def move(self, n):

        temp = n % len(self)

        return self[temp : ] + self[ : temp]

if __name__ == '__main__':

    a = MoveableStr("abcde")

    print(a.move(3)) # 输出 deabc

    print(a.move(-3)) # 输出 cdeab
```

9. 请用继承的方法完成习题 6 的要求。

【参考答案】

```
class Point:

    def __init__(self, x, y):

        self.x = x

        self.y = y

    def __str__(self):

        return 'Point('+str(self.x)+','+str(self.y)+')'

class Point1(Point):

    def __add__(self, other):

        return Point1(self.x+other.x, self.y+other.y)
```

```
def __sub__(self, other):  
    return Point1(self.x-other.x, self.y-other.y)  
  
def __mul__(self, n):  
    return Point1(self.x*n, self.y*n)  
  
def __truediv__(self, n):  
    return Point1(self.x/n, self.y/n)  
  
if __name__ == '__main__': # 这是测试代码  
    a,b=Point1(4,5),Point1(2,3)  
    print(a+b) # 输出结果为'Point(6,8)'  
    print(a-b) # 输出结果为'Point(2,2)'  
    print(a*2) # 输出结果为'Point(8,10)'  
    print(a/2) # 输出结果为'Point(2.0,2.5)'
```

10.考虑如下一个简单的图形绘制程序。定义一个 **Point** 类表示二维平面上的点（可以借鉴第 6 题）。所有图形实体的基类为 **Shape**，其包含一个 **Point** 类型的 **position** 实例属性，用于表示图形的位置。**Shape** 类有一个方法 **move_to** 将图形的位置移动到新的位置，**shape** 还包括一个 **draw** 方法，打印输出 **Shape** 对象的字符串表示。继承 **Shape** 类的具体图形类型包括线段类 **Line** 和圆类 **Circle**。**Line** 类初始化的第一个参数为一个端点，也是其位置，另一个参数表示另一个端点，作为另一个实例属性。**Circle** 类初始化的第一个参数表示其圆心，也是其位置，另一个参数表示其半径，作为另一个实例属性。如下的代码已经给出了 **Shape** 类的定义，请完成 **Point** 类、**Line** 类和 **Circle** 类的定义。**Line** 类需要重写 **move_to** 方法，在移动位置的同时修改另一个端点的坐标。另外，**Line** 类和 **Circle** 类都要求对 **__str__** 重写实现，以得到更直观的 **draw** 效果，其中类 **Line** 的输出信息样式为“Line:第一个端点的坐标--第二个端点的坐标”，类 **Circle** 的输出信息样式为“Circle center:圆心坐标,R=半径”。部分例程和测试代码如下：

请实现类 Point、Line、Circle

```
class Shape:
```

```
    # 不要修改该类
```

```
    def __init__(self):
```

```
        self.position = Point(0,0)
```

```
    def draw(self):
```

```
        print(str(self))

    def move_to(self,position):

        print("moving to ",str(position))

        self.position = position

# 下面是测试代码

if __name__ == '__main__':

    l = Line(Point(0,0),Point(20,20))

    l.draw()

    l.move_to(Point(3,5)) # 移动到一个新的点

    l.draw()

    c = Circle(Point(10,10),5)

    c.draw()

    c.move_to(Point(30,20))

    c.draw()

# 下面是脚本的期望输出

Line:(0,0)----(20,20)

moving to  (3,5)

Line:(3,5)----(23,25)

Circle center:(10,10),R=5

moving to  (30,20)

Circle center:(30,20),R=5
```

【参考答案】

```
class Point:

    def __init__(self,x,y):

        self.x = x

        self.y = y

    def __str__(self):

        return '('+str(self.x)+','+str(self.y)+')'

    def __add__(self,other):
```

```
        return Point(self.x+other.x,self.y+other.y)

    def __sub__(self,other):
        return Point(self.x-other.x,self.y-other.y)

    def __mul__(self,n):
        return Point(self.x*n,self.y*n)

    def __truediv__(self,n):
        return Point(self.x/n,self.y/n)

class Shape:
    def __init__(self):
        self.position = Point(0,0)
    def draw(self):
        print(str(self))
    def move_to(self,position):
        print("moving to ",str(position))
        self.position = position

class Line(Shape):
    def __init__(self,position,end):
        self.position = position
        self.end = end

    def move_to(self,position):
        deltap = position - self.position
        super().move_to(position)
        self.end += deltap
```

```
def __str__(self):  
    return  
'Line:'+str(self.position)+'----'+str(self.end)  
  
class Circle(Shape):  
    def __init__(self,center,radius):  
        self.position = center  
        self.radius = radius  
    def __str__(self):  
        return 'Circle  
center:'+str(self.position)+',R='+str(self.radius)  
  
if __name__ == '__main__':  
    l = Line(Point(0,0),Point(20,20))  
    l.draw()  
    l.move_to(Point(3,5)) # 移动到一个新的点  
    l.draw()  
    c= Circle(Point(10,10),5)  
    c.draw()  
    c.move_to(Point(30,20))  
    c.draw()
```

附录 1:任课教师介绍



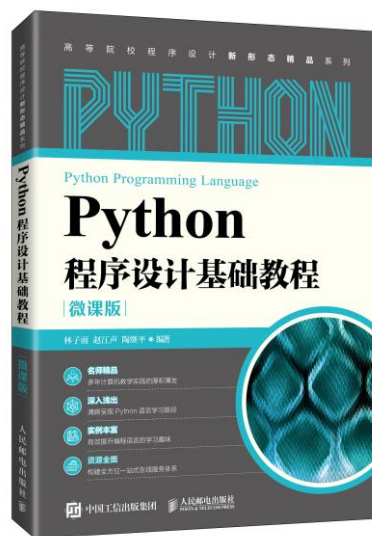
林子雨，男，博士（毕业于北京大学），厦门大学计算机科学系副教授，厦门大学信息学院实验教学中心主任。2013 年度、2017 年度和 2020 年度厦门大学教学类奖教金获得者。中国计算机学会数据库专业委员会委员，中国计算机学会信息系统专业委员会委员，厦门大学数据库实验室负责人，数据中国“百校工程”教育部专家组成员。负责的“大数据技术原理与应用”课程获评“2018 年国家精品在线开放课程”和“2020 年国家级线上一流本科课程”。负责的“Spark 编程基础”课程获评“2020 年福建省线上一流本科课程”。国内高校首个“数字教师”的提出者和建设者，编著出版了国内高校第一本系统介绍大数据知识的专业教材《大数据技术原理与应用》，成为国内众多高校开课教材，同时建设了国内高校首个大数据课程公共服务平台，为教师教学和学生学大数据课程免费提供全方位、一站式服务，平台累计访问量超过 1000 万次，成为国内高校大数据教学知名品牌，并获得“2018 年福建省教学成果二等奖”和“2018 年厦门大学教学成果特等奖”。

E-mail: ziyulin@xmu.edu.cn

个人主页: <http://dblab.xmu.edu.cn/linziyu>

数据库实验室网站: <http://dblab.xmu.edu.cn>

附录 2：课程教材介绍



出版社：人民邮电出版社

ISBN: 978-7-115-57519-7 定价：59.8 元

出版时间：2021 年 1 月第 1 版

本书详细介绍了获得 Python 基础编程能力所需要掌握的各方面技术。全书共 15 章，内容包括 Python 语言概述、基础语法知识、程序控制结构、序列、字符串、函数、面向对象程序设计、模块、异常处理、基于文件的持久化、基于数据库的持久化、图形用户界面编程、正则表达式、网络爬虫、常用的标准库和第三方库等。本书每个章节都安排了入门级的编程实践操作，以便读者更好地学习和掌握 Python 编程方法。本书官网免费提供了全套的在线教学资源，包括讲义 PPT、习题、源代码、软件、数据集、上机实验指南等。



扫一扫访问教材官网